

Chat Bridge

Architecture & file reference — the backend-for-frontend between the Angular chat UI and the MCP Gateway

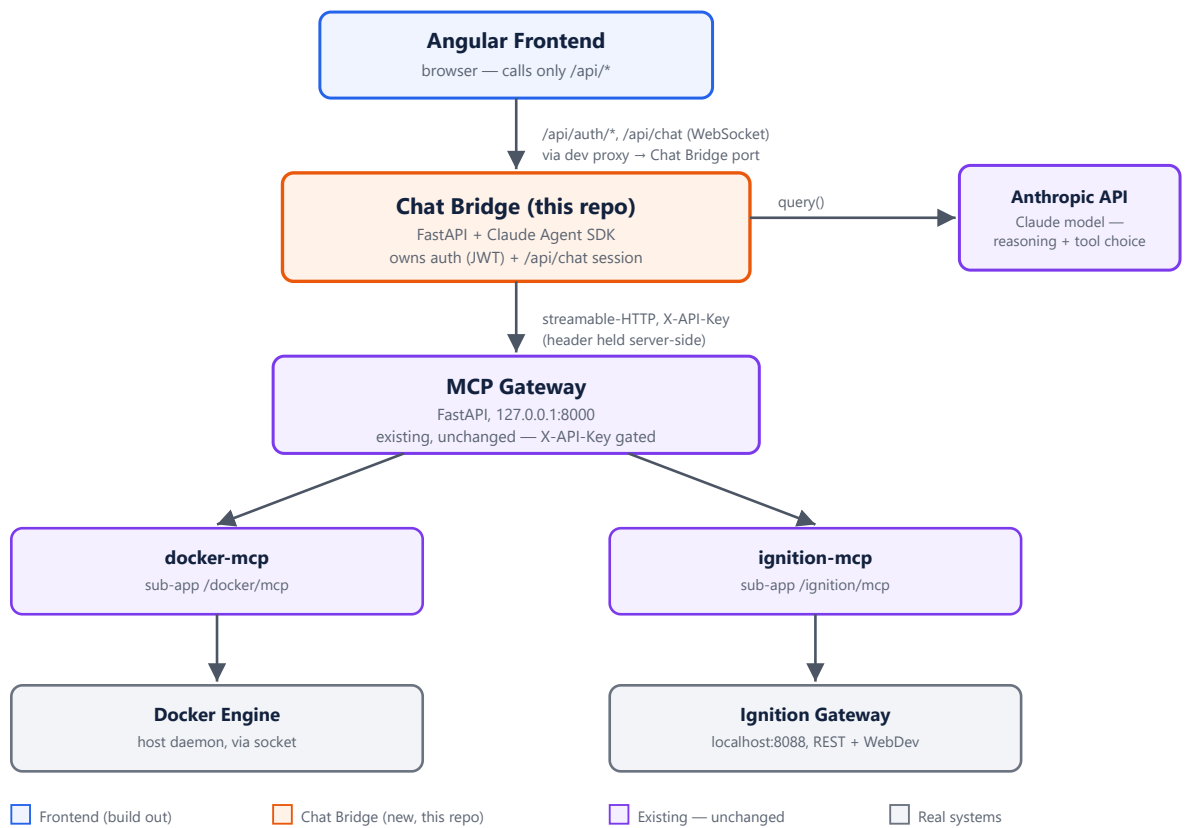
Prepared 2026-08-13 · C:\Users\katlego.gagoopane\Documents\2026\Claude\chat-bridge

1. What Chat Bridge Does

Chat Bridge is a standalone Python service — FastAPI plus the Claude Agent SDK — that sits between the Angular frontend and the existing MCP Gateway. It is a **backend-for-frontend (BFF)**: the browser talks to it and nothing else, and it in turn talks to both Anthropic's API (for Claude's reasoning) and the MCP Gateway (for the actual Docker and Ignition tool calls).

This design was chosen deliberately over an alternative: the Angular project already ships an empty Express skeleton under `frontend/server/`. That skeleton is intentionally left unused — Chat Bridge takes over the entire backend role instead, including end-user authentication, so there is exactly one browser-facing service and one place that holds the MCP Gateway's API key.

2. System Architecture



The MCP Gateway is never reached directly from the browser — the Angular app's own `/api`-only rule is preserved end to end because Chat Bridge is the single public entry point. The frontend's Express skeleton (`frontend/server/`) does not

appear in this diagram because it plays no role in the request path.

3. Request Flow for a Single Chat Turn

What happens, end to end, when a user sends one message in the chat UI (auth steps are the planned addition — see §6):

- 1 Browser opens `/api/chat` over the Angular dev proxy; the proxy forwards the WebSocket upgrade to Chat Bridge.
- 2 Chat Bridge's `app.py` accepts the connection and opens one `ChatSession` (`claude_service.py`) for the lifetime of that connection — one `ClaudeSDKClient` per browser tab.
- 3 The browser sends a JSON frame `{"content": "..."};` `ChatSession.send()` calls `client.query(prompt)`.
- 4 The Claude Agent SDK sends the conversation plus the tool list to **Anthropic's API**. The model decides what to say and, if a Docker or Ignition action is needed, which MCP tool to call.
- 5 For a tool call, the SDK calls the **MCP Gateway** directly over streamable-HTTP, at `GATEWAY_URL/docker/mcp` or `GATEWAY_URL/ignition/mcp`, with the `X-API-Key` header that `claude_service.py` attached from `config.py`'s settings — the browser never sees this key.
- 6 The Gateway's API-key middleware checks the header, then routes to the matching sub-app (`docker-mcp` or `ignition-mcp`), which executes the real action against the Docker daemon or the Ignition Gateway.
- 7 The result flows back up: Gateway → SDK → Claude (which turns it into a reply, possibly after further tool calls) → `ChatSession.send()`, which yields each `SDKMessage` as it arrives.
- 8 `app.py` serializes each message (`serialize_message` / `_serialize_block` in `claude_service.py`) and pushes it to the browser as a JSON WebSocket frame — so text, tool calls, and tool results stream in as they happen rather than waiting for the full answer.

4. Project Layout — File by File

`src/chat_bridge/app.py` ENTRYPOINT

The FastAPI application. Exposes `GET /health` (returns `{"status": "ok"}`), used to confirm the service is up) and the `WebSocket /api/chat` endpoint. On each new WebSocket connection it opens a `ChatSession`, then loops: receive a JSON frame from the browser, forward its `content` to the session, stream every resulting message back as JSON. Currently **unauthenticated** by design — JWT verification is added once the login/signup endpoints exist (see §6).

src/chat_bridge/claude_service.py CORE LOGIC

Where Chat Bridge talks to both Anthropic and the MCP Gateway. `build_options()` constructs a `ClaudeAgentOptions` whose `mcp_servers` point at the Gateway's `/docker/mcp` and `/ignition/mcp` streamable-HTTP endpoints, each with the `X-API-Key` header attached server-side, and whose `allowed_tools` auto-approve both servers (`mcp_docker_*`, `mcp_ignition_*`) — there is no interactive terminal on this end of the connection to answer a permission prompt. `ChatSession` wraps one `ClaudeSDKClient` for a connection's lifetime (`connect()` on entry, `disconnect()` on exit) and exposes `send(prompt)`, an async generator that calls `client.query()` then yields every message from `client.receive_response()`, serialized to plain dicts by `serialize_message` / `_serialize_block`.

src/chat_bridge/config.py CORE LOGIC

A `pydantic-settings` `Settings` class, instantiated once as the module-level `settings` object and imported wherever configuration is needed. Reads from a local `.env` (or the process environment): `gateway_url`, `gateway_api_key` (required — the process fails fast at import time if it's missing), `host` / `port` (aliased to `CHAT_BRIDGE_HOST` / `CHAT_BRIDGE_PORT`), and an optional `claude_model` override.

src/chat_bridge/__init__.py ENTRYPOINT

Defines `main()`, the function the `chat-bridge` console script (declared in `pyproject.toml`) runs: starts `uvicorn` against `chat_bridge.app:app`, bound to `settings.host` / `settings.port`.

pyproject.toml CONFIG

Project metadata and dependencies, in the same `uv`-managed style as `backend/mcp-gateway`. Runtime dependencies: `fastapi`, `uvicorn[standard]`, `claude-agent-sdk`, `pydantic-settings`. Declares the `chat-bridge` console script and a `uv_build` build backend.

.env.example CONFIG

Template for the real `.env` (gitignored, not yet created). Documents every variable `config.py` reads — `GATEWAY_URL`, `GATEWAY_API_KEY` (must match the Gateway's own key), `CHAT_BRIDGE_HOST` / `PORT`, `CLAUDE_MODEL` — plus a note that the Claude Agent SDK reads `ANTHROPIC_API_KEY` straight from the process environment, not from this file.

.python-version META

Pins the interpreter to `3.12` for `uv`, matching the rest of the backend.

.gitignore META

Standard Python ignores (`__pycache__` / build artifacts, `.venv`) plus `.env`, so the real Gateway API key and JWT secret never get committed.

uv.lock GENERATED

Resolved dependency lockfile, written by uv sync . Not hand-edited.

ARCHITECTURE.md DOCS

The architecture decision record this PDF is generated from: why Chat Bridge exists as a separate BFF, what it owns, why the frontend's Express skeleton stays dormant, the v3 plan's build order, and the open questions listed in §7 below.

5. Configuration Reference

Variable	Read by	Purpose
GATEWAY_URL	config.py	Base URL of the MCP Gateway (default http://127.0.0.1:8000)
GATEWAY_API_KEY	config.py	Required. Must match the Gateway's own GATEWAY_API_KEY; sent as X-API-Key
CHAT_BRIDGE_HOST	config.py	Bind address for this service (default 127.0.0.1)
CHAT_BRIDGE_PORT	config.py	Bind port (default 8001) — the Angular dev proxy's /api target repoints here
CLAUDE_MODEL	config.py	Optional model alias override (e.g. sonnet, opus)
ANTHROPIC_API_KEY	claude-agent-sdk directly	Not read from .env — must be set in the process environment (or a logged-in Claude Code CLI)

6. Current Status

Chat Bridge is scaffolded through step 1 of the frontend integration plan (frontend/mcp_frontend_v3.pdf):

Step	Status
1. Stand up the Chat Bridge — FastAPI + Claude Agent SDK, MCP servers wired to the Gateway's /docker and /ignition, API key held server-side	Done
2. Add auth endpoints (/api/auth/login, /api/auth/signup), mock user, JWT issuance	Not started
3. Add JWT verification to the /api/chat WebSocket	Not started
4. Repoint the Angular dev proxy's /api target to this service	Not started
5–7. Angular auth screens, chat UI, routing	Not started
8–9. End-to-end + edge-case testing	Not started

10. Deploy (compose Chat Bridge + Gateway; only the Bridge's port reaches the browser)

Not
started

Verified so far: `uv sync` installs cleanly, all SDK names `claude_service.py` imports exist in the installed `claude-agent-sdk` version, and the app boots and answers `GET /health`.

7. Open Questions

- JWT client storage — httpOnly cookie set by the Bridge vs. a token attached manually to the WebSocket upgrade.
- Whether the mock in-memory user model is permanent or a placeholder for real user storage later.
- Per-tool confirmation UX for high-impact Docker/Ignition actions — `claude_service.py` currently auto-approves both tool servers wholesale.
- Whether conversation history should persist beyond a single WebSocket connection.